Design Pattern Analysis and Evolution in the JUnit 6 Testing Framework

23CSE455 Design Patterns – Case study, team B16

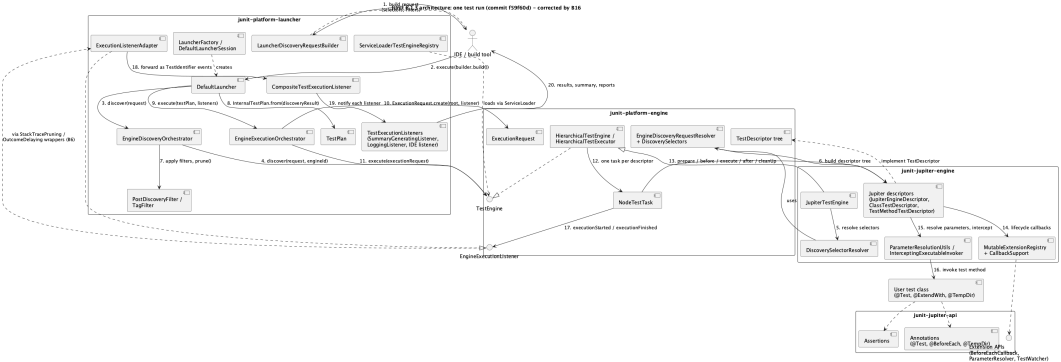
Gautham Mohanraj (AM.SC.U4CSE23319, CSE D)

2026-10-10

Application and scope

- JUnit 6.1.3, commit `f59f60d`, Eclipse Public License 2.0
- Production code of 4 modules: platform-engine, platform-launcher, jupiter-api, jupiter-engine
- Four areas: (A) discovery & descriptors, (B) engines & lifecycle, (C) listeners & reporting, (D) Jupiter extensions & resources
- Every pattern claim checked in the source; line ranges computed from the code, never typed

Architecture of one test run

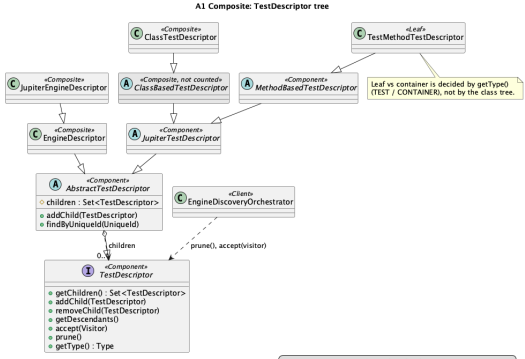


Part 1: patterns found at the baseline

Pattern	Instances
Strategy	10
Adapter	5
Observer	5
Decorator	4
Template Method	3
Composite	2
Command	2
Null Object	2
Chain of Responsibility	1
Factory Method	1
Total	**35**

- 10 pattern types, 35 instances
- 76 concrete participant classes, 152 unique symbols
- 25 rejected, 8 uncertain candidates
- Composite rule: combine children of the same interface *and* merge their answers
- 102 LLM claims verified

Example: A1 Test Descriptor Tree (Composite)



Part 2: three changes

Change	What	Kind	Refactoring
CH03 filter policy	`off` | `exclude` | `only` over tags/methods from `junit-platform.properties`	implementation change	preparatory Extract Method
CH01 summary listener	per-class table + slowest tests, ServiceLoader, off by default	existing extension point	Extract Method `count()` (CCN 10 → 4)
CH02 `@ManagedResource`	create/inject/destroy any resource, per class or per test	existing extension points only	none (no existing code changed)

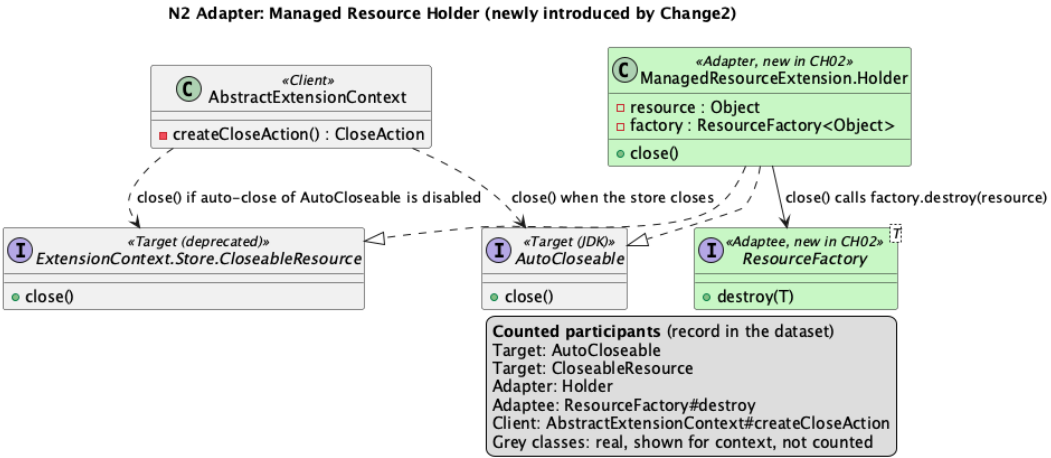
- JUnit suites after every commit: launcher 382 → 397, platform 2452 → 2467, Jupiter 3381 → 3388, 0 failures
- Mutation checks: the new tests fail when the feature is removed

Pattern evolution

Snapshot	Instances	Concrete	Change
Original	35	76	baseline
After CH03	35	78	A2 Composite, A3 Strategy *extended*
After CH01	35	79	C1 Observer *extended*
After CH02	37	81	D3 *extended*; N1 Strategy, N2 Adapter *new*

- No pattern replaced or removed: changes plug into existing Strategy/Observer points
- LLM corrections: no "enum Strategy" (D1-18), not a Long Method (D1-20), Holder *is* an Adapter (D1-22)

N2: Managed Resource Holder (Adapter, new in CH02)



Part 3: all 10 pattern types in four languages

Language	Mean LOC	Max CC	Median ms
Java	138.0	6–22	32.0–36.8
Python	86.5	6–22	21.0–21.8
JavaScript	102.0	5–17	91.4–93.2
C++	142.1	6–24	4.8–5.9

40 programs; ranges over the 10 problems.

- One problem per type, each modelled on a Part 1 instance (A1, A3, B1, B2, B7, B9, C1, C7, D9, D10)
- Same input, byte-identical output; 110/110 commands pass; every runner 3/3; C++ clean under ASan/UBSan, 0 leaks
- Java/C++ state contracts in types; Python/JS rely on duck typing
- C++ makes ownership explicit (`unique_ptr`); time is runtime start-up

AI use and verification

- claude.ai: analysis prompts, 2 Part 3 problems; Claude Code: Part 2 code, 8 Part 3 problems, scripts (all in `data/llm.csv`)
- Verified claims: Part 1: 102, Part 2: 22, Part 3: 16 – with corrections recorded, not hidden
- Generated code accepted only after JUnit's own suites, mutation checks and demos on the official vs evolved jar

Findings

- JUnit is built on a few patterns used many times: Strategy for every policy, Observer for events, Adapter between APIs, Composite for the test tree
- These extension points made two of three changes possible without touching JUnit's code
- Refactoring helped where the smell was real (CCN 10 $\rightarrow$ 4) and changed little where it was preparatory
- Limits: four modules, production code only, single-machine timings